

Reflektion

Under detta projekt har jag lärt mig mycket om AWS och hur molninfrastruktur fungerar i verkligheten. Jag har också lärt mig hur olika AWS-tjänster kan arbeta tillsammans för att bygga en modern webbapplikation.

Projektet hjälpte mig att förstå skillnader mellan olika arkitekturer, säkerhet i AWS och hur resurser kommunicerar med varandra.

Två olika infrastrukturer

I projektet använde jag två olika infrastrukturer:

- Första infrastrukturen: byggd med ECS/EC2 och Docker-containerar.
- Andra infrastrukturen: byggd med serverless-tjänster som Lambda och API Gateway.

När jag arbetade med båda lösningarna såg jag att de har olika styrkor och svagheter. Vilken lösning som är bäst beror på vilken typ av applikation man utvecklar.

Containerbaserad lösning

Den containerbaserade lösningen gav mig större kontroll över backend-applikationen och miljön där den kördes. Spring Boot-applikationen kunde köras i en Docker-container och sedan deployas till EC2 genom ECS.

Fördelar med containerbaserad lösning

- Applikationen fungerade likadant i olika miljöer.
- Containerar gjorde deployment enklare och mer organiserad.
- Jag fick mer kontroll över backend och servermiljö.

Nackdelar med containerbaserad lösning

- ECS krävde ganska mycket konfiguration.
- Jag behövde hantera task definitions, nätverk, portar och Security Groups.
- Flera gånger fick jag problem med anslutningar.
- Vissa containerar startade inte som de skulle.

Detta visade att containerbaserad arkitektur ger stor flexibilitet men kräver också mer kunskap om AWS-infrastruktur och nätverk.

Serverless-lösning

Den andra lösningen byggdes med AWS Lambda och API Gateway.

I denna lösning behövde jag inte hantera servrar manuellt. Lambda-funktioner användes för enklare uppgifter, till exempel API-anrop och skapande av pre-signed URLs för uppladdning av filer till S3.

Fördelar med serverless

- Enkel deployment.
- AWS skalar funktionerna automatiskt.
- Bra för mindre och separata funktioner.
- Lägre kostnad eftersom resurser används bara vid behov.

Nackdelar med serverless

- Det blev svårare att hålla struktur när antalet Lambda-funktioner ökade.
- Debugging och övervakning blev mer komplicerat.
- Loggar och funktioner blev utspridda på flera ställen.

Säkerhet och access

Under projektet lärde jag mig också hur resurser skyddas i AWS.

IAM-roller användes för att ge rätt behörigheter till olika resurser. Till exempel kunde backend och Lambda få tillgång till S3 utan att använda access keys direkt i koden.

Security Groups fungerade som en brandvägg. De bestämde vilken trafik som fick komma in eller ut från resurserna.

Databasen i RDS var privat och kunde bara nås från backend-servern. Detta gjorde systemet säkrare eftersom databasen inte var öppen mot internet.

Frontend i S3 och CloudFront var publik eftersom användare behövde kunna öppna webbplatsen från internet. Samtidigt blockerades direkt access till S3-bucketen genom Origin Access Control (OAC), vilket ökade säkerheten.

Sammanfattning

Sammanfattningsvis har projektet gett mig bättre förståelse för AWS, cloud computing och säkerhet i molnet. Jag har lärt mig skillnader mellan containerbaserad arkitektur och serverless-lösningar samt hur viktigt det är med rätt permissions, nätverk och säkerhet.

Det mest utmanande under projektet var att konfigurera nätverk och permissions korrekt. Små fel i IAM eller Security Groups kunde göra att tjänster inte fungerade tillsammans. Samtidigt var detta också den mest lärorika delen av projektet.